

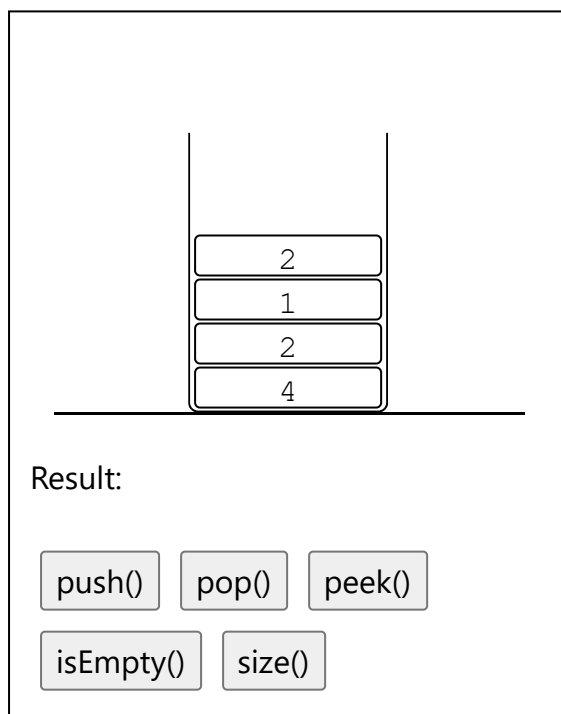
ADVERTISEMENT

DSA Stacks

[< Previous](#)[Next >](#)

Stacks

A stack is a data structure that can hold many elements.



Think of a stack like a pile of pancakes.

In a pile of pancakes, the pancakes are both added and removed from the top. So when removing a pancake, it will always be the last pancake you added. This way of organizing elements is called LIFO: Last In First Out.

Basic operations we can do on a stack are:



Size Finds the number of elements in the stack.

Experiment with these basic operations in the stack animation above.

Stacks can be implemented by using arrays or linked lists.

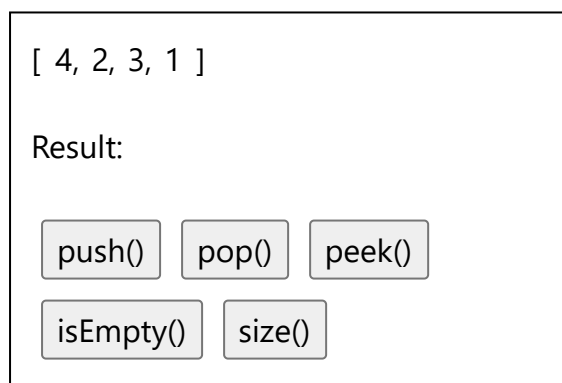
Stacks can be used to implement undo mechanisms, to revert to previous states, to create algorithms for depth-first search in graphs, or for backtracking.

Stacks are often mentioned together with Queues, which is a similar data structure described on the next page.

Stack Implementation using Arrays

To better understand the benefits with using arrays or linked lists to implement stacks, you should check out [this page](#) that explains how arrays and linked lists are stored in memory.

This is how it looks like when we use an array as a stack:



Reasons to implement stacks using arrays:

- **Memory Efficient:** Array elements do not hold the next elements address like linked list nodes do.
- **Easier to implement and understand:** Using arrays to implement stacks require less code than using linked lists, and for this reason it is typically easier to understand as well.

A reason for **not** using arrays to implement stacks:

- **Fixed size:** An array occupies a fixed part of the memory. This means that it could take up more memory than needed, or if the array fills up, it cannot hold more elements.



ADVERTISEMENT

Example

Python:

```
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, element):
        self.stack.append(element)

    def pop(self):
        if self.isEmpty():
            return "Stack is empty"
        return self.stack.pop()

    def peek(self):
        if self.isEmpty():
            return "Stack is empty"
        return self.stack[-1]

    def isEmpty(self):
        return len(self.stack) == 0

    def size(self):
        return len(self.stack)

# Create a stack
myStack = Stack()

myStack.push('A')
myStack.push('B')
myStack.push('C')
print("Stack: ", myStack.stack)

print("Pop: ", myStack.pop())

print("Peek: ", myStack.peek())
```



ADVERTISEMENT

[Run Example »](#)

Stack Implementation using Linked Lists

A reason for using linked lists to implement stacks:

- **Dynamic size:** The stack can grow and shrink dynamically, unlike with arrays.

Reasons for **not** using linked lists to implement stacks:

- **Extra memory:** Each stack element must contain the address to the next element (the next linked list node).
- **Readability:** The code might be harder to read and write for some because it is longer and more complex.

This is how a stack can be implemented using a linked list.

Example

Python:

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

class Stack:
    def __init__(self):
        self.head = None
        self.size = 0

    def push(self, value):
        new_node = Node(value)
        if self.head:
```



```

def pop(self):
    if self.isEmpty():
        return "Stack is empty"
    popped_node = self.head
    self.head = self.head.next
    self.size -= 1
    return popped_node.value

def peek(self):
    if self.isEmpty():
        return "Stack is empty"
    return self.head.value

def isEmpty(self):
    return self.size == 0

def stackSize(self):
    return self.size

myStack = Stack()
myStack.push('A')
myStack.push('B')
myStack.push('C')

print("Pop: ", myStack.pop())
print("Peek: ", myStack.peek())
print("isEmpty: ", myStack.isEmpty())
print("Size: ", myStack.stackSize())

```

[Run Example »](#)

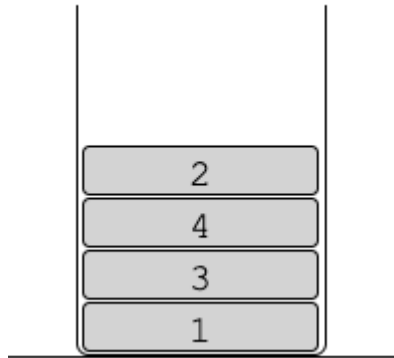
DSA Exercises



ADVERTISEMENT

Exercise:

The image below represents a "Stack" data structure.



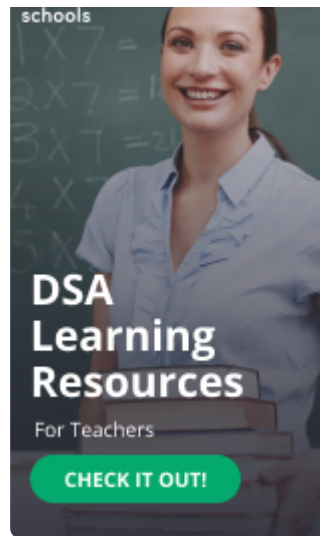
Running the `peek()` method on the Stack above, what is returned?

[Submit Answer »](#)[Start the Exercise](#)[◀ Previous](#)[Sign in to track progress](#)[Next >](#)

ADVERTISEMENT

[Tutorials ▼](#)[References ▼](#)[Exercises ▼](#)[Sign In](#)[W3.CSS](#) [C](#) [C++](#) [C#](#) [BOOTSTRAP](#) [REACT](#) [MYSQL](#) [JQUERY](#) [EJS](#)

ADVERTISEMENT



COLOR PICKER



ADVERTISEMENT

ADVERTISEMENT



PLUS

SPACES

GET CERTIFIED

FOR TEACHERS



Tutorials ▼

References ▼

Exercises ▼



Sign In

[W3.CSS](#)
[C](#)
[C++](#)
[C#](#)
[BOOTSTRAP](#)
[REACT](#)
[MYSQL](#)
[JQUERY](#)
[EJS](#)

Top Tutorials

[HTML Tutorial](#)
[CSS Tutorial](#)
[JavaScript Tutorial](#)
[How To Tutorial](#)
[SQL Tutorial](#)
[Python Tutorial](#)
[W3.CSS Tutorial](#)
[Bootstrap Tutorial](#)
[PHP Tutorial](#)
[Java Tutorial](#)
[C++ Tutorial](#)
[jQuery Tutorial](#)

Top References

[HTML Reference](#)
[CSS Reference](#)
[JavaScript Reference](#)
[SQL Reference](#)
[Python Reference](#)
[W3.CSS Reference](#)
[Bootstrap Reference](#)
[PHP Reference](#)
[HTML Colors](#)
[Java Reference](#)
[AngularJS Reference](#)
[jQuery Reference](#)

Top Examples

[HTML Examples](#)
[CSS Examples](#)
[JavaScript Examples](#)
[How To Examples](#)
[SQL Examples](#)
[Python Examples](#)
[W3.CSS Examples](#)
[Bootstrap Examples](#)
[PHP Examples](#)
[Java Examples](#)
[XML Examples](#)
[jQuery Examples](#)

Get Certified

[HTML Certificate](#)
[CSS Certificate](#)
[JavaScript Certificate](#)
[Front End Certificate](#)
[SQL Certificate](#)
[Python Certificate](#)
[PHP Certificate](#)
[jQuery Certificate](#)
[Java Certificate](#)
[C++ Certificate](#)
[C# Certificate](#)
[XML Certificate](#)


[FORUM](#)
[ABOUT](#)
[ACADEMY](#)

W3Schools is optimized for learning and training. Examples might be simplified to improve reading and learning.

Tutorials, references, and examples are constantly reviewed to avoid errors, but we cannot warrant full correctness

of all content. While using W3Schools, you agree to have read and accepted our [terms of use](#), [cookie and privacy policy](#).

[Tutorials ▼](#)[References ▼](#)[Exercises ▼](#)[Sign In](#)[≡](#) [W3.CSS](#) [C](#) [C++](#) [C#](#) [BOOTSTRAP](#) [REACT](#) [MYSQL](#) [JQUERY](#) [EJS](#)

ADVERTISEMENT